Trie and Trie Again: The Quest to GPU Parallelize MDTRIE

Micah Gold
micah.gold@yale.edu

Advisor: Anurag Khandelwal
anurag.khandelwal@yale.edu

A Senior Thesis as a partial fulfillment of requirements for the Bachelor of Science in Computer Science

Department of Computer Science
Yale University
April 30, 2026

Acknowledgements

I would like to express my sincere gratitude to Professor Anurag Khandelwal for his guidance and mentorship throughout this thesis. His expertise, thoughtful feedback, and unwavering support were instrumental in shaping this work from its earliest stages to its completion. I am also deeply grateful to Yash Lala and Leo Li for their patience and collaboration over the course of this project. Their willingness to engage, discuss, and work through challenges together made this research far richer than it could have been otherwise. Finally, I would like to thank the Yale Department of Computer Science for its support of the BS/MS program, which provided me with the academic foundation, resources, and community necessary to pursue this work.

Contents

1 Introduction **6**

2 Background **8**
- 2.1 Retrieval-Augmented Generation 8
- 2.2 From FAISS to MDTRIE 8
- 2.3 MDTRIE and the TRINITY System 8
- 2.4 GPU Implementation Challenges as Design Constraints 9

3 Methodology **10**
- 3.1 Range-Search Algorithms and Implementations 10
  - 3.1.1 Serial Range-Search Model 10
  - 3.1.2 Parallel Range Search Model 11
  - 3.1.3 Implementation Details for CPU and Single-Thread GPU Range Search 11
  - 3.1.4 Implementation Details for the Optimized GPU Range Search . . 12
- 3.2 Evaluation Methodology 13
  - 3.2.1 Hardware and Software Platform 13
  - 3.2.2 Dataset and Query Construction 14
  - 3.2.3 Correctness Validation 14
  - 3.2.4 Metrics and Instrumentation 14

4 Results **15**
- 4.1 Single-Query Latency Across Dimensions 15
- 4.2 Architectural Overhead of a Naive GPU Port 16
- 4.3 Optimized Single-Query and Batched Kernels 17
- 4.4 Trie Memory and the d Ceiling 18
- 4.5 Summary 18

5 Related Work **21**
- 5.1 Approximate Nearest-Neighbor Search for RAG 21
- 5.2 GPU Acceleration of Tree and Trie Structures 21
  - 5.2.1 NOVA Lab MDTRIE Research 22

6 Conclusions **23**

7 Future Work **24**
7.1 Hierarchical Encodings for High-Dimensional Data 24
7.2 Dimension Partitioning for SM-Level Parallelism 24

A Appendix **28**

Trie and Trie Again: The Quest to GPU Parallelize MDTRIE

Micah Gold

Abstract

Retrieval-augmented generation (RAG) pipelines perform many k-nearest-neighbor (KNN) lookups under strict latency budgets. Production systems rely on memory-intensive, GPU-accelerated libraries such as FAISS to meet these demands.

MDTRIE, a succinct multi-dimensional trie optimized for RAG, offers an appealing alternative: it is self-indexed and maintains low storage overhead. However, its sequential KNN lookup cannot compete with GPU-accelerated systems, and no GPU implementation has previously been attempted.

This thesis presents the first GPU implementation of MDTRIE and characterizes the architectural barriers it faces. We develop three range-search implementations over a common simplified MDTRIE: a CPU baseline, a single-thread GPU port that preserves the baseline's logic, and an optimized GPU kernel built on a split-stream, per-layer trie representation that assigns one cooperating thread block per query. On an NVIDIA RTX A5000, we evaluate these implementations across indexed point counts from 10^3 to 2×10^5 and projected dimensionalities $d \in \{4, 8, 12, 16, 20\}$. The single-thread GPU port is 27–62$\times$ slower than the CPU baseline, a gap we attribute to unhidden global-memory latency rather than arithmetic cost. The optimized single-query GPU kernel outperforms the baseline at large N, and a batched variant achieves roughly $70\times$ speedup over the baseline at $N = 2\times10^5$ and $d = 4$. However, the simplified structure's 2^d-bit uncollapsed bitmap exhausts device memory by $d = 20$, making representation design, not kernel tuning, the binding constraint for embedding-scale workloads. Future work should consider how to employ some form of hierarchical node representation in MDTRIE without hindering GPU parallelization.

1 Introduction

Large Language Models (LLMs) are context machines. They ingest large sequences of input data, tokenize it into ”context”, and exploit learned relationships between tokens to produce useful outputs.

The quantity and relevance of the context provided to an LLM directly determines output quality: richer context yields more accurate, coherent, and task-aware generations. In recent years, companies have raced to expand the size of context windows—the quantity of information an LLM can keep directly available to influence its next output. Today, leading models support million-token context windows. Modern applications push LLMs to operate over entire codebases, lengthy document corpora, and hours of live-streaming video. Unfortunately, context windows remain too small to hold and use all of this data at once.

Modern deployments address this gap with *retrieval-augmented generation* (RAG). In RAG, a query (or partial conversation state) is embedded into a vector space and used to retrieve the most relevant documents or embeddings from an external store, supplying the model with targeted context at generation time [1].

In production RAG systems, LLMs execute many nearest-neighbor lookups under strict latency budgets, often in large batches. While GPUs are a natural fit for the dense linear algebra of embedding models, the retrieval step is frequently bottlenecked by index traversal, memory bandwidth, and irregular control flow. Industry-grade GPU retrieval libraries such as FAISS demonstrate that high-throughput similarity search is possible, but they typically rely on substantial auxiliary indexing (e.g., inverted lists, quantization structures, graph neighborhoods) that trade memory for speed [2]. In many settings, including edge deployments and tightly budgeted inference stacks, index memory overhead is not “free”.

This project studies an alternative design point: compressed, trie-based candidate generation using MDTRIE, the dynamic succinct multi-dimensional data structure introduced in TRINITY [3]. MDTRIE combines a generalized Morton-code representation with a self-indexed succinct trie layout that enables fast pruning for multi-attribute range searches and efficient point queries [3]. TRINITY shows that this design can deliver low-latency lookups with low storage overhead on CPUs. To remain competitive under the performance demands of production RAG pipelines, an MDTRIE would need to be parallelized. Yet no GPU implementation of MDTRIE has been attempted or even proposed. Perhaps this is because the core advantage of MDTRIE—bit-packed traversal with aggressive pruning—is also what makes it difficult to parallelize: the computation is branchy, bit-manipulation heavy, and sensitive to memory layout and divergence.

The primary technical goal of this work is to introduce a GPU-parallel range-search kernel for MDTRIE to support high-throughput candidate generation for vector embed-

ding retrieval, and to evaluate whether MDTRIE's compression and pruning benefits can be retained under GPU constraints (memory coalescing, warp divergence, and tail latency) [4, 5].

2 Background

2.1 Retrieval-Augmented Generation

Retrieval-augmented generation supports LLMs by granting them on-demand access to external knowledge. A retrieval module queries a vector database and injects the most relevant documents into the model's context window before output generation [1]. In the dominant "dense retrieval" variant, both queries and documents are embedded into a shared vector space, and retrieval reduces to a nearest-neighbor search. For our purposes, we formalize this as a bounding-region query. Given a query vector $q \in \mathbb{R}^d$ we retrieve all indexed points within a radius $r \in \mathbb{R}$.

2.2 From FAISS to MDTRIE

GPU-accelerated vector similarity search is a well-studied problem. Facebook Artificial Intelligence Similarity Search (FAISS) demonstrates that careful kernel design and disciplined use of the GPU memory hierarchy can improve RAG throughput [2]. Yet this performance comes at a cost. High-throughput approximate nearest-neighbor pipelines typically layer additional index structures and multi-stage filtering on top of raw vector storage, resulting in substantial memory overhead.

MDTRIE takes the opposite approach. Rather than augmenting a flat vector store with auxiliary indices, it organizes data in a compressed, self-indexed representation that subsumes both storage and search structure. It uses far less memory than FAISS. The trade-off is traversal cost. MDTRIE encodes its index as a succinct trie stored in bit-packed form. Traversal over this representation resists parallelization in a fundamental way. Each traversal step depends on the outcome of the last, making it difficult to dispatch independent threads without stalling or diverging [3].

2.3 MDTRIE and the TRINITY System

TRINITY introduces MDTRIE as a dynamic succinct multi-dimensional data structure designed for multi-attribute range search and efficient point queries [3]. Morton codes (Z-order curves) map multi-dimensional points to a single linearized dimension while partially preserving spatial locality. Naive range search over Morton intervals is nonetheless expensive because Z-order curves contain discontinuities that force traversal into regions geometrically distant from the query box. MDTRIE resolves this by keying points on a Morton encoding and using the trie structure to prune irrelevant Morton-prefix regions

during traversal [3].

MDTRIE is optimized to be space efficient. The trie is encoded as a compact bit vector. Nodes are stored in depth-first-order. Nodes with one child are represented in **collapsed** form, compressing the space required to store their children from 2^n bits down to n [3]. Other nodes are represented as small-sub-trees themselves so-called **hierarchical** form. To handle high dimensionality and dynamic updates, MDTRIE partitions the trie into fixed-size **treeblocks** connected by frontier pointers [3]. Together these optimizations tighten the memory profile and improve sequential traversal time. However, they also add implicit dependencies between steps in the traversal process. Because nodes are of variable size and there are few pointers between them, a thread cannot jump directly to the next node without first interrogating the current one. This makes MDTRIE traversal difficult to parallelize.

2.4 GPU Implementation Challenges as Design Constraints

The CUDA programming model is built around massive data parallelism, but its execution model imposes strict constraints on how that parallelism must be organized. Threads are grouped into warps that execute a single instruction in lockstep. When threads within a warp diverge, the hardware serializes both paths and discards the work of idle threads [4]. Memory bandwidth is most efficiently utilized when threads in a warp access a contiguous, aligned region of global memory in a single transaction. Pointer chasing and scattered bit-vector probes break coalescing and can quickly become the dominant bottleneck [5].

For a succinct structure like MDTRIE, the constraints of the GPU programming model force two questions. First, what is the right unit of parallelism? Should threads be assigned to independent queries, treeblocks, frontier nodes, or individual traversal steps? Second, how must the representation itself change, in its bit layout, node grouping, and auxiliary metadata, so that parallel traversal produces coalesced memory access rather than scattered probes?

3 Methodology

3.1 Range-Search Algorithms and Implementations

We implement three versions of MDTRIE and three corresponding versions of range search. The **CPU implementation** is the baseline: it establishes the traversal semantics and provides the reference output for correctness checking. The **single-threaded GPU implementation** is the diagnostic: it preserves the same search logic while moving execution onto the GPU, allowing the cost of translation alone to be measured. The **optimized GPU implementation** is the final target: it changes both the representation and the execution model so that the same range-search problem is expressed in a form that matches GPU hardware.

3.1.1 Serial Range-Search Model

From a theoretical standpoint, the CPU baseline range search and the single-threaded GPU range search are the same.

They both run on a simplified version of MDTRIE (Figure 3.1). In this version of MDTRIE, there is no pointer chasing, no treeblocks, and no hierarchical nodes.

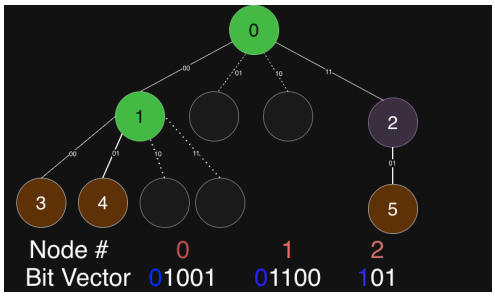


Figure 3.1: MDTRIE structure. Each internal node stores a child bitmap indicating which of the 2^d symbols are present. Node 2 is collapsed (degree one) and stores its single symbol directly rather than a full bitmap. Leaves (brown) correspond to stored Morton codes.

Each internal node corresponds to one Morton symbol and stores either a child bitmap or, if the node has degree one, a collapsed symbol. Nodes in the simplified MDTRIE bitmap are stored in breadth-first "level-order". In this way the simplified MDTRIE differs from the one employed in TRINITY, which uses depth-first "pre-order". Importantly, the traversal algorithm itself is agnostic to the node ordering method. We chose to use BFS for the baseline because we wanted to minimize the differences between the CPU and GPU implementations and BFS ordering simplifies the GPU programming model (see Section 3.1.3).

During traversal, the query is a box in the projected coordinate space, represented as a pair of Morton-space bounds $(q_{\min}, q_{\max})$. At level l, the traversal extracts the admissible symbol interval $[s_\ell^{\min}, s_\ell^{\max}]$ from those bounds, examines the node's children, and determines whether they must be searched or whether they can be pruned. If they must

be searched they are added to a frontier queue for processing. A frontier entry stores the current node, the Morton prefix accumulated so far, and the tightened query bounds inherited from its parent. When a child with symbol s is kept, the bounds for that child are shrunk so that all coordinates agree with the bits fixed by s at level ℓ. By the time a leaf node is traversed, the Morton code has been fully determined by the search bounds, and is emitted. This algorithm is exact and deterministic, so it serves as a natural baseline for both correctness and single-query runtime.

3.1.2 Parallel Range Search Model

The optimized GPU algorithm solves the same search problem but leverages data parallelism. Instead of asking one thread to walk the trie serially, it assigns one thread block to one query. The algorithm maintains a frontier array which holds the nodes in the current layer that must be traversed. At start time, the frontier is populated with the root node. At each step, the algorithm processes all nodes in the frontier in parallel and adds their children to the next frontier if they must be searched.

To support this execution pattern, the trie is transformed into a split-stream, per-layer representation by a pre-processing step. For each depth ℓ, the optimized structure stores an indicator bit-vector marking collapsed versus uncollapsed nodes, an exclusive scan $U[\ell]$ mapping each node to its uncollapsed rank, the compact child bitmaps for uncollapsed nodes, the collapsed symbols for collapsed nodes, and an exclusive scan $P[\ell]$ of child counts, which maps a node to the BFS position of its first children in the next layer. This organization makes child localization constant-stride and layer-local.

For uncollapsed nodes, the optimized kernel computes an admissible-symbol mask rather than testing each present child independently. The mask combines two constraints: the symbol must lie inside $[s_\ell^{\min}, s_\ell^{\max}]$, and it must match every Morton-prefix bit already fixed by the current bounds. Intersecting that mask with the node's child bitmap yields exactly the children that survive to the next frontier. The work is therefore shifted from repeated per-child predicate evaluation to bitmap arithmetic, sparse iteration, and cooperative output reservation.

3.1.3 Implementation Details for CPU and Single-Thread GPU Range Search

The CPU implementation and the single-thread GPU implementation intentionally share the same logical structure. They each possess two arrays:

1. `child_bitmap`: the BFS ordered node bitmap
2. `node_first_child`: the BFS index of each node's first child.

The CPU baseline uses this layout directly (Algorithm 1). The single-thread GPU implementation uploads the same flat trie to device memory and launches one kernel per query. The kernel uses the same breadth-first frontier expansion, the same collapsed-node fast path, and the same Morton-bound tightening logic. The differences are architectural

Algorithm 1 CPU range search for one query (breadth-first)

Require: Flat trie arrays $\{\texttt{child_bitmap}, \texttt{node_first_child}\}$, query bounds $(\texttt{q_start}, \texttt{q_end})$
Ensure: All Morton codes that fall inside the query box

Initialize frontier $F_0 \leftarrow \{(\text{root}, \emptyset, \texttt{q_start}, \texttt{q_end})\}$
for $\ell = 0$ to 31 **do**
  $F_{\ell+1} \leftarrow \emptyset$
  for all entries $(\texttt{node}, \texttt{code}, \texttt{start}, \texttt{end})$ in F_ℓ **do**
    Extract admissible symbol interval $[s_\ell^{\min}, s_\ell^{\max}]$ from `start`, `end` at depth ℓ
    if $s_\ell^{\min} > s_\ell^{\max}$ **then continue**
    end if
    $\texttt{bound_magic} \leftarrow \neg(s_\ell^{\min} \oplus s_\ell^{\max}) \wedge \texttt{kSymbolMask}$
    if `node` is collapsed **then**
      Read its single symbol s
      if s matches `bound_magic` and $s \in [s_\ell^{\min}, s_\ell^{\max}]$ **then**
        Tighten $(\texttt{start}, \texttt{end})$ and append child to $F_{\ell+1}$
      end if
    else
      $\texttt{rank} \leftarrow 0$
      for all present-child bits s in node's bitmap (ascending order) **do**
        if s matches `bound_magic` and $s \in [s_\ell^{\min}, s_\ell^{\max}]$ **then**
          Tighten bounds; append $(\texttt{first_child} + \texttt{rank}, \texttt{code} \mid s, \texttt{start}', \texttt{end}')$ to $F_{\ell+1}$
        end if
        $\texttt{rank} \leftarrow \texttt{rank} + 1$
      end for
    end if
  end for
end for
Emit the accumulated Morton codes from the final frontier

rather than algorithmic: the logic is identical, but the execution context is not. Every memory access on the GPU comes from device memory, and because only one thread is active, the kernel cannot hide global-memory latency behind other ready warps.

3.1.4 Implementation Details for the Optimized GPU Range Search

The optimized implementation introduces both a new layout and a new kernel driver. The layout is the split-stream representation described above, materialized as per-layer arrays `indicator`, `U`, `uncollapsed_bitmaps`, `collapsed_symbols`, and `P`. Query bounds are decoded once into per-dimension ordered `uint32` values and then carried in frontier entries as $\texttt{lo}[d]$ and $\texttt{hi}[d]$, which makes bound tightening a small number of bitwise operations instead of repeated Morton-code reconstruction.

The kernel assigns one block to one query. Within a layer, threads cooperatively

Algorithm 2 Optimized GPU range search for one query

Require: Per-layer arrays $\{\texttt{indicator}, \texttt{U}, \texttt{uncollapsed_bitmaps}, \texttt{collapsed_symbols}, \texttt{P}\}$, query bounds $(\texttt{lo}, \texttt{hi})$
Ensure: All Morton codes that fall inside the query box

Initialize frontier $F_0 \leftarrow \{(\text{root}, \texttt{lo}, \texttt{hi})\}$
for $\ell = 0$ to 31 **do**
    $F_{\ell+1} \leftarrow \emptyset$
    for all frontier entries in F_ℓ processed cooperatively by the block **do**
        Extract the admissible symbol interval $[s_\ell^{\min}, s_\ell^{\max}]$
        if node is collapsed **then**
            Read its single symbol from `collapsed_symbols`
            if that symbol satisfies the current bounds **then**
                Tighten $(\texttt{lo}, \texttt{hi})$ and append one child to $F_{\ell+1}$
            end if
        else
            Read the node bitmap from `uncollapsed_bitmaps`
            Build a bitmap of in-range child symbols
            Use warp scans to reserve output slots for surviving children
            Tighten bounds and append each surviving child to $F_{\ell+1}$
        end if
    end for
end for
Decode the final frontier entries back into Morton codes and emit them

process frontier entries, compute per-node survivor counts, use warp scans to reserve output slots, and append surviving children into the next frontier. Synchronization occurs only at layer boundaries. This preserves the dependency structure of trie traversal while still extracting the available intra-query parallelism.

The paper's main evaluation focuses on this optimized single-query kernel. A batched wrapper that launches multiple queries concurrently also exists, but it is not the central claim of the paper. Its role is best understood as future headroom: once a single query is expressed efficiently, independent queries can be scheduled together with little conceptual difficulty.

3.2 Evaluation Methodology

3.2.1 Hardware and Software Platform

All experiments were run on an NVIDIA RTX A5000 using CUDA 13.0 and driver 580.65.06. The benchmark harness reports that a GPU SM clock of 1.695 GHz and a CPU clock of 2.200 GHz were maintained throughout the runs.

Table 3.1: Experimental platform and benchmark configuration.

GPU	NVIDIA RTX A5000 (CUDA 13.0, driver 580.65.06)
CPU clock reported by harness	2.200 GHz
GPU SM clock reported by harness	1.695 GHz
Projected trie dimensionality	4-D from 32-D float32 records
Queries per case	128 deterministic radius queries
Largest trie evaluated	200,000 projected points

3.2.2 Dataset and Query Construction

The harness operates on a 32-dimensional float32 embedding file but projects each record onto a fixed 4-dimensional subspace, namely coordinates $\{0, 1, 2, 3\}$. This keeps the evaluation centered on one succinct trie, which is the unit of traversal studied in this paper. For each case size $N \in \{10^2, 10^3, 5 \times 10^3, 10^4, 2 \times 10^4, 5 \times 10^4, 10^5, 2 \times 10^5\}$, the harness samples N records deterministically, builds the trie, and then evaluates 128 deterministic radius queries with radius 0.1 in the projected space.

3.2.3 Correctness Validation

Every GPU result is compared against the CPU baseline output set before its timing is accepted. This is important for two reasons. First, it ensures that the single-thread GPU diagnostic really is a semantics-preserving translation of the CPU code. Second, it ensures that the optimized GPU kernel is evaluated as an exact search algorithm rather than as a potentially lossy approximation. The results therefore compare implementations of the same exact range-search task.

3.2.4 Metrics and Instrumentation

The benchmark exports several classes of measurements: Morton encode and decode time on both CPU and GPU, per-query single-query range-search latency for all three implementations, and storage for both trie layouts. For the single-thread GPU diagnostic it additionally records detailed kernel counters, including nodes popped, symbols scanned, and time buckets for bounds arithmetic, collapsed-node handling, uncollapsed scans, and residual work. These measurements are especially useful because they let the paper distinguish between two explanations for poor direct GPU performance: host-device overhead versus work actually spent inside the translated kernel.

4 Results

4.1 Single-Query Latency Across Dimensions

Figure 4.1 shows per-query range-search latency at $\mathrm{d} = 4$ for all three implementations as the number of indexed points grows from 1000 up to the largest trie that fit in device memory. Three features of this plot drive the rest of the analysis.

First, the CPU baseline is fast in absolute terms: at $N = 200{,}000$ points it answers a single radius-0.1 query in roughly 0.107 ms, and it stays well under a millisecond across every trie size we tested. Second, the single-thread GPU diagnostic is substantially slower than CPU at every scale, despite executing the same algorithm: at the same $N = 200{,}000$ point the direct GPU translation takes 2.85 ms per query, a factor of $\sim 27\times$ slower. Moving the same traversal onto one GPU thread does not buy anything because the kernel cannot hide global-memory latency and cannot exploit any intra-query parallelism. Third, for small numbers of points ($N < 100{,}000$) the single-query optimized GPU kernel performs much better than the single-threaded implementation, but still slightly worse than the CPU implementation. It only outperforms our CPU implementation when our data structure contains large numbers of points. Luckily, almost all vector databases used in RAG have upwards of $100{,}000$ data points and most have millions. For these large datasets, our GPU kernel appears to be the way to go.

Of course, if we allow batching, our optimized kernel beats the CPU baseline dramatically even for small numbers of points. At $N = 200{,}000$ its per-query cost drops to $1.5\,\mu$s, roughly $70\times$ faster than the serial CPU baseline, by folding many queries into a single grid launch and amortizing launch overhead across thousands of cooperating threads.

Figures 4.2a–4.2d repeat the CPU and single-thread GPU comparison at $\mathrm{d} \in \{8, 12, 16, 20\}$. These figures therefore isolate a different question: does the CPU vs. single-thread GPU gap persist as the symbol alphabet grows? The answer is yes, and it widens with dimensionality. At the largest trie we could build for each configuration, the CPU-to-GPU latency ratio grows from $\sim 27\times$ at $\mathrm{d} = 4$ to $\sim 26\times$ at $\mathrm{d} = 8$, $\sim 28\times$ at $\mathrm{d} = 12$, $\sim 47\times$ at $\mathrm{d} = 16$, and $\sim 62\times$ at $\mathrm{d} = 20$, consistent with increased per-node memory traffic as the uncollapsed bitmap grows. The slight non-monotonicity at $\mathrm{d} = 8, 12$ reflects variance in tree shape across configurations; the overall trend is strongly upward in d. The $\mathrm{d} = 20$ run also exhausted device memory beyond $5{,}000$ indexed points, the first concrete symptom of the 2^{d} representation limit discussed in Section 4.4.

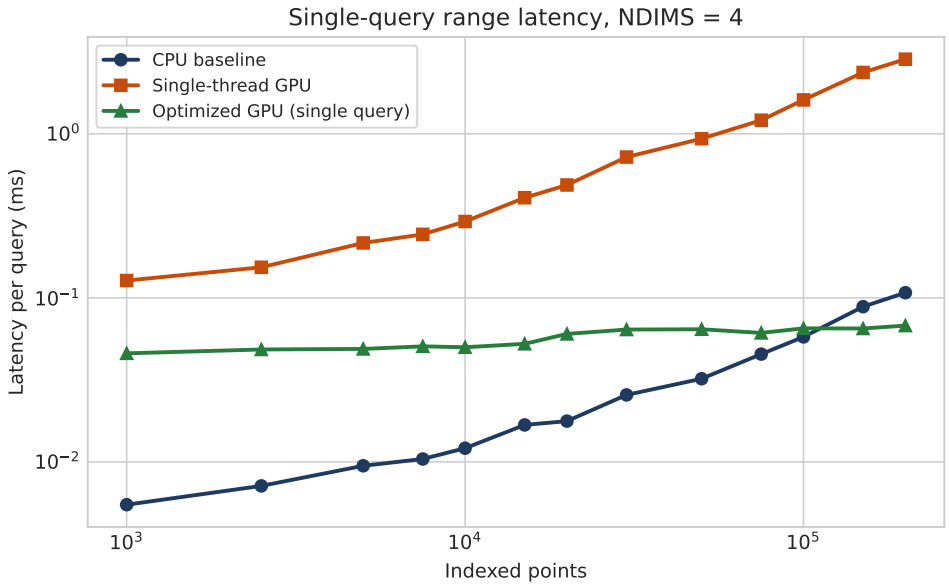
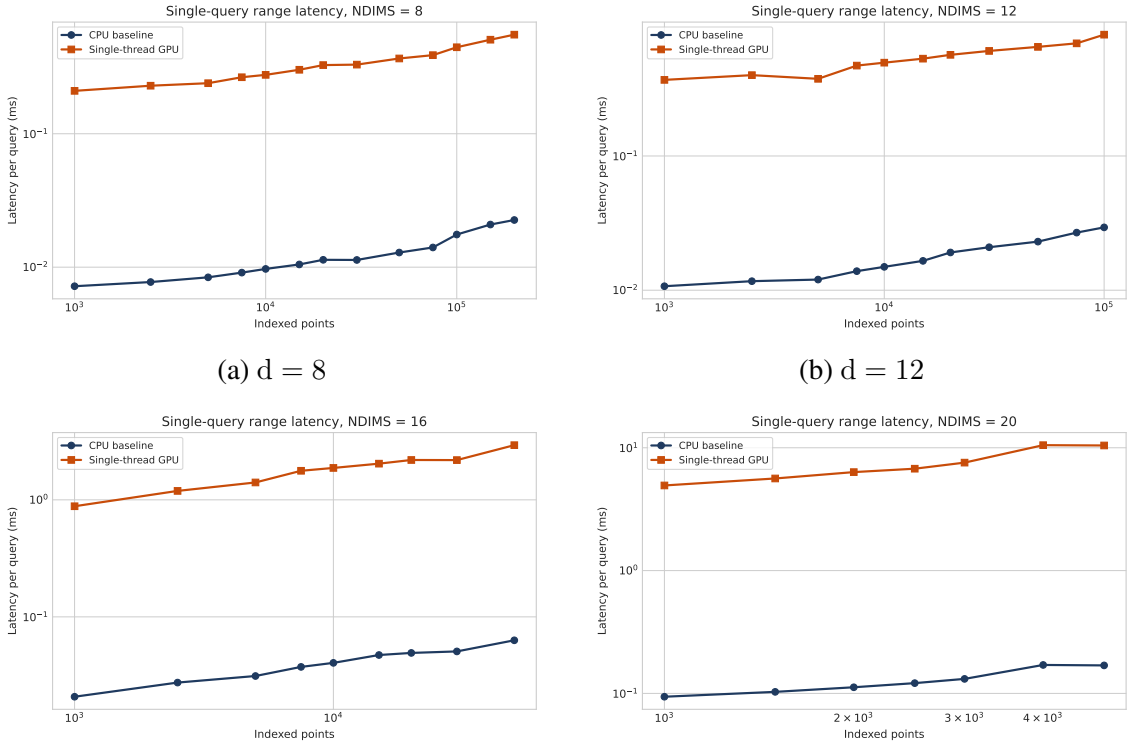


Figure 4.1: Single-query range-search latency for CPU, single-thread GPU, and the optimized split-stream GPU kernel at $\mathrm{d} = 4$. Both axes are log-scaled. The CPU baseline remains below one millisecond at every scale; the single-thread GPU translation grows with trie size because each traversal step must cross global memory; the optimized single-query GPU kernel reduces but does not eliminate this gap.

4.2 Architectural Overhead of a Naive GPU Port

Figure 4.3 plots the CPU-to-single-thread-GPU latency ratio across all tested dimensions and point counts. A ratio greater than one means the CPU is faster than a direct translation of the same algorithm onto a single GPU thread. The ratio is above one at every (N, d) point we tested, and it grows with the size of the trie. As the trie gets larger, the gap between the CPU baseline and the single-thread GPU diagnostic widens.

This is the core diagnostic claim of the paper. Nothing about "single-thread" means "slow" in the abstract: a single GPU thread executes a comparable number of arithmetic operations per clock as a CPU thread. What the single-thread GPU kernel cannot do is hide global-memory latency behind other ready warps or reuse data out of a well-populated L1. Each traversal step reads a bit-vector from device memory, makes a short arithmetic decision, and fetches the next node. With no other active warps to cover the memory stalls, the cost of a single query grows with the number of nodes touched, which is itself roughly the log-depth of the trie times the average surviving frontier width. For more insight into why the GPU single-threaded version is so much slower than the CPU version, see the Appendix.



(a) $\mathrm{d} = 8$ (b) $\mathrm{d} = 12$

(c) $\mathrm{d} = 16$ (d) $\mathrm{d} = 20$

Figure 4.2: Single-query range-search latency across higher d values. Each subfigure shows the CPU baseline and the single-thread GPU diagnostic. Both implementations follow the same scaling trend (roughly linear in $\log N$), separated by a nearly constant multiplicative gap—the single-thread GPU port is not paying an asymptotic penalty, just a large constant per-access one.

4.3 Optimized Single-Query and Batched Kernels

The optimized split-stream kernel addresses the memory-latency problem in two ways. At the single-query level, it allocates one block per query and lets the threads in that block cooperate on frontier expansion, so that bitmap fetches and scans run in parallel and collapsed-vs-uncollapsed work overlaps across lanes. At the batched level, it launches all Q queries as independent blocks in a single grid, so the GPU schedules many warps concurrently and the latency of any one query's bitmap fetch is hidden behind work from the other queries.

Figure 4.4 compares the batched optimized kernel to the CPU baseline at $\mathrm{d} = 4$. The batched per-query time (total kernel time divided by Q) sits between 1.1 and $1.5\,\mu$s across the full range $N \in [10^3, 2\times10^5]$, while the CPU baseline rises from $5.8\,\mu$s at $N = 1{,}000$ to $107\,\mu$s at $N = 200{,}000$. The batched kernel is therefore both absolutely faster than CPU at every scale and roughly flat in N, because the GPU's SM count, not the per-query frontier, is the dominant constraint on throughput once enough queries are in flight.

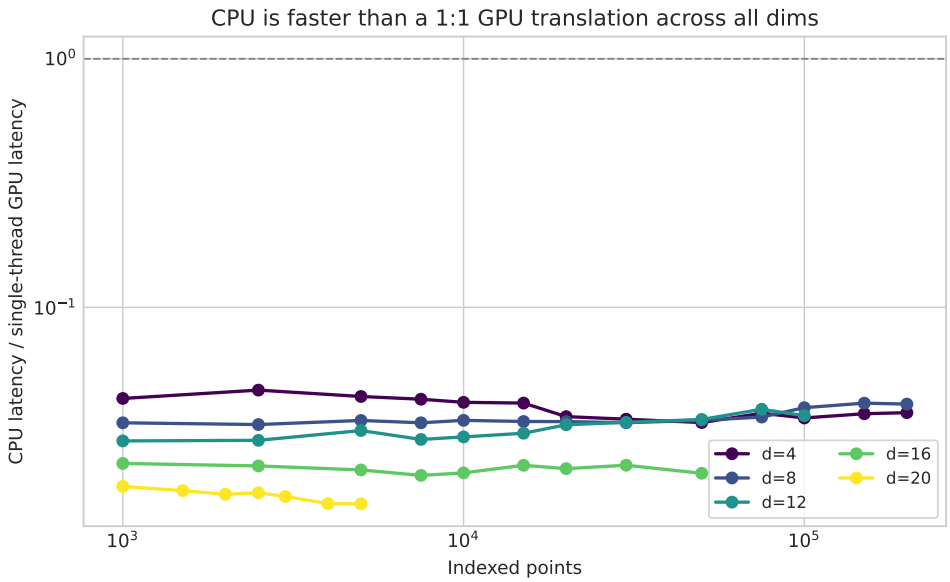


Figure 4.3: Ratio of CPU latency to single-thread GPU latency across all tested d and point counts. Values above one mean the CPU is faster than the same algorithm run serially on a single GPU thread. The ratio grows with trie size at every dimensionality, showing that the single-thread port's overhead is dominated by unhidden global-memory latency rather than per-node arithmetic.

4.4 Trie Memory and the d Ceiling

Figure 4.5 shows how the flat trie footprint grows with N at each d we were able to benchmark. For fixed N, footprint grows approximately as 2^{d} per uncollapsed node because each uncollapsed child bitmap has 2^{d} bits by construction. The practical consequence is visible in our sweep: at $\mathrm{d} = 20$ the first $(N = 10{,}000)$ case already fails with "invalid argument" when the driver is asked to upload the trie, because the flat bitmap storage alone exceeds the A5000's usable device memory budget. This is the same structural constraint that motivates the hierarchical-encoding future work discussed in Section 7.1.

4.5 Summary

Three empirical claims carry through the rest of the paper.

1. A direct port of the CPU range-search kernel to a single GPU thread is slower than the CPU baseline at every dimensionality we tested (Figures 4.1, 4.2, and 4.3). The gap widens with trie size, which is consistent with unhidden global-memory latency rather than excess arithmetic work.

2. The split-stream optimized kernel closes this gap and then some, but only once queries are batched so the grid has enough independent blocks to saturate the SMs

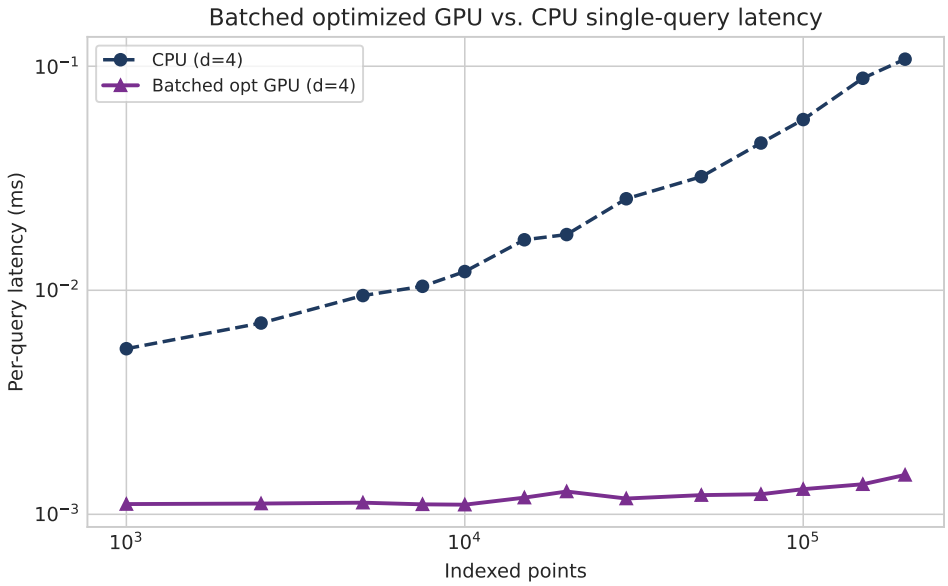


Figure 4.4: Per-query latency of the batched optimized GPU kernel vs. the serial CPU baseline at $\mathrm{d} = 4$. The batched value is the total kernel event time divided by the number of concurrent queries in the grid launch, so it reflects GPU-wide throughput rather than serial per-query latency.

(Figure 4.4). Single-query traversal on the GPU is intrinsically memory-bound, which is exactly why a batched driver is necessary.

3. The flat 2^{d}-bit bitmap representation cannot scale to arbitrary dimensionality without a hierarchical reorganization of the trie (Figure 4.5). This is both a practical limit on the present implementation and the direct motivation for the hierarchical-encoding future work.

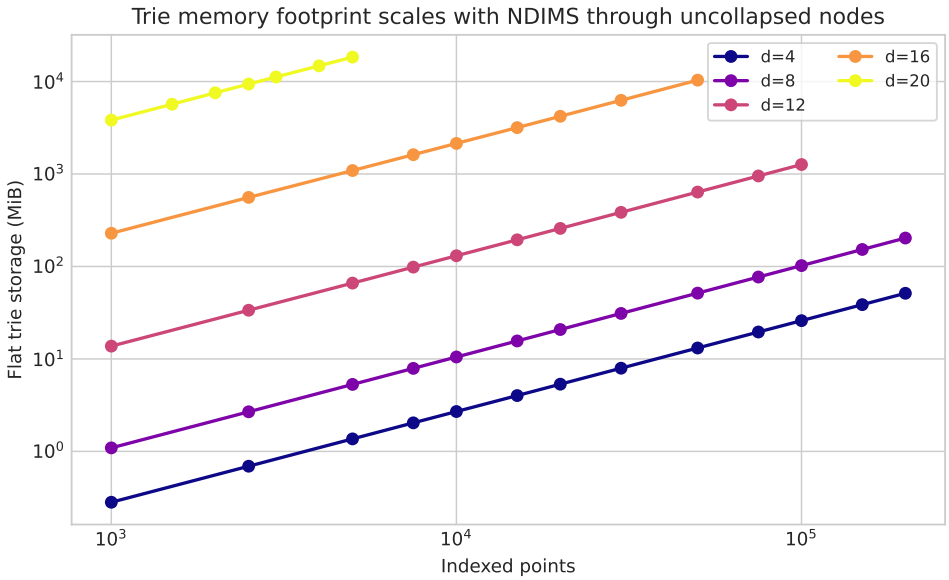


Figure 4.5: Flat trie storage as a function of indexed points for each evaluated d value. Both axes are log-scaled. The per-dimension curves separate vertically because the uncollapsed child bitmap has 2^{d} bits per internal node that branches: widening the symbol alphabet multiplies the per-node footprint even when the number of nodes is similar.

5 Related Work

5.1 Approximate Nearest-Neighbor Search for RAG

Vector retrieval at scale is dominated by approximate nearest-neighbor search over learned embeddings. Graph-based methods such as HNSW [6] construct hierarchical proximity graphs that support logarithmic-expected-time search, and have become the de facto standard in vector databases including Milvus, Weaviate, and Qdrant. Quantization-based methods, exemplified by IVF-PQ [7] and its descendants, partition the space with inverted lists and compress residuals via product quantization to trade recall for memory footprint. FAISS [2] combines these techniques and provides highly optimized GPU kernels for both brute-force and IVF-style search, demonstrating that billion-scale similarity search is feasible on commodity accelerators. ScaNN [8] refines quantization with anisotropic loss functions tuned for maximum inner-product search. These systems achieve impressive throughput, but as mentioned earlier their speed is predicated on auxiliary structures—coarse quantizers, proximity graphs, and learned codebooks—whose memory overhead is often several multiples of the raw vector data. In memory-constrained settings such as on-device inference or tightly budgeted serving stacks, this overhead is a meaningful cost.

5.2 GPU Acceleration of Tree and Trie Structures

As discussed earlier, parallelizing pointer-based data structures on GPUs is notoriously difficult because of warp divergence and uncoalesced memory access. Prior work has examined GPU implementations of B-trees [9], radix trees [10], and learned indexes [11], with varying success depending on query regularity. For spatial search specifically, GPU k-d tree traversal [12] and bounding-volume hierarchies from ray tracing provide useful templates, but both assume low-dimensional geometry. Succinct structures pose additional challenges: rank-select operations on bitvectors are bit-manipulation heavy and branchy, and traversal decisions depend on data loaded only one level deep, limiting the scope for prefetching. Parallel wavelet tree construction work [13] has shown that succinct operations can be made throughput-competitive when batched carefully, though GPU-specific realizations of these ideas remain nascent. To our knowledge, no prior work has attempted a GPU implementation of a multi-dimensional succinct trie, and in particular none has targeted MDTRIE.

5.2.1 NOVA Lab MDTRIE Research

This thesis extends an ongoing line of MDTRIE research at Yale's NOVA Lab that started with the TRINITY paper. Recently, Yash Lala and Leo Li have been investigating MDTRIE's behavior in high-dimensional embedding spaces, where the curse of dimensionality degrades the pruning effectiveness. Their solution is to introduce tiling-based radius estimation for k-nearest-neighbor queries, alongside a suite of traversal optimizations that reduce the effective depth of the trie during search. Their work establishes that MDTRIE can be made competitive with graph-based ANN methods on CPUs for high-dimensional embedding workloads.

Another Senior Thesis presented this Spring by Wesely Andrade investigates compressing MDTRIE into a directed acyclic word graph (DAWG) representation. In DAWGs, points that share runs of bits (e.g. prefixes, suffixes) share nodes in the graph. For typical workloads, Andrade has shown that DAWG MDTRIE takes up considerably less space than MDTRIE while achieving comparable lookup performance.

Our contribution is orthogonal to both efforts. We focus on the GPU parallelization of the core range-search primitive on the standard MDTRIE representation, and we view our kernel as a foundation on top of which Lala and Li's pruning heuristics and Andrade's DAWG compression could later be layered. Nevertheless, we acknowledge that integration with their innovations will be nontrivial. For example, DAWG MDTRIE is likely more difficult to traverse efficiently in parallel. This is because in a DAWG, leaf nodes may be reached through multiple paths, so a single thread in a warp would not have sufficient information to prune its child's subtree.

6 Conclusions

The goal of this thesis was to develop a performant, parallel MDTRIE range search algorithm for accelerating RAG. We showed that while a naive GPU single-threaded translation of MDTRIE range search is not performant, a co-designed GPU implementation, by contrast, can speed up queries on large datasets. Our main contribution was the design and benchmarking of a highly-tuned GPU kernel for parallel range search, which is available at our public project repository.

Notably, because MDTRIE's uncollapsed child bitmap has 2^{NDIMS} bits per node, the data structure used in this analysis cannot be pushed to the dimensionalities that matter for real embedding workloads. The device-memory ceiling is hit at $\mathrm{NDIMS} = 20$ with only $10{,}000$ indexed points. Closing this gap requires changing the representation itself, which is the subject of the hierarchical-encoding and dimension-partitioning directions outlined in Chapter 7.

Overall, GPU-parallel MDTRIE range search is possible, it is fast where the structure's representational assumptions hold, and its remaining obstacles are structural rather than implementation-specific. The path from here to an embedding-scale GPU MDTRIE is one of representation design under the constraints this thesis has measured and characterized, not one of further low-level kernel tuning.

7 Future Work

7.1 Hierarchical Encodings for High-Dimensional Data

All three of our range search implementations were benchmarked on a simplified MDTRIE data structure. As discussed previously, this simplified version of MDTRIE was employed because it did not require pointer chasing (which is slow on GPUs) for traversal.

Unfortunately, in this simplified MDTRIE, a single uncollapsed node takes up more than 2^d bits. This is tractable for $d = 4$, where a node fits in a single word, but for $d = 16$ the bitmap already spans $65{,}536$ bits per node, and for $d = 64$ a single node requires more than 2 exabytes of memory!

For context, all state-of-the-art RAG systems use embedding dimensions upwards of 384 and some use as many as 4096 [14], so actually using the simplified MDTRIE for RAG use cases is impossible.

The natural solution to the storage problem is developing a pointer-less hierarchical encoding scheme for uncollapsed nodes. Such a scheme would allow the trie to save memory by discarding large continuous regions of a node that contain no descendants. The issue is that any hierarchical encoding necessarily sequentializes the process of scanning an uncollapsed node and processing its children.

The central question for this line of work is whether the GPU speedups reported here can survive a transition to any feasible hierarchical encoding scheme. Answering this question is necessary before MDTRIE-style GPU range search can be claimed as a general tool for high-dimensional data rather than a specialized result for small d.

7.2 Dimension Partitioning for SM-Level Parallelism

The current optimized kernel assigns one thread block—and therefore one streaming multiprocessor (SM)—to one query. Within each layer, threads in the block cooperatively process the frontier. The layers themselves are traversed sequentially, and the work of a single query is confined to a single SM regardless of how much of the GPU is otherwise idle.

A promising extension is to partition the d-dimensional MDTRIE into k sub-tries, each handling d/k dimensions per level, and to assign each sub-trie to its own SM. Concretely, at every depth each sub-trie consumes a disjoint slice of the Morton symbol and fixes only the bits corresponding to its assigned dimensions. A query then executes as k cooperating traversals, one per SM, whose surviving frontiers are intersected at layer boundaries to produce the full admissible set. Because each sub-trie operates on a narrower symbol alphabet ($2^{d/k}$ rather than 2^d), its per-node bitmaps are smaller, its

admissible-symbol masks are cheaper to construct, and its frontier expansions touch less memory per step.

Right now, it is unclear whether the cost of running set intersection at each layer would harm performance. Still, if the intersection can be implemented as bitmap arithmetic on compact frontiers, the scheme may plausibly trade a modest synchronization cost for a factor-k reduction in per-SM work and a corresponding increase in aggregate GPU utilization.

Bibliography

[1] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7 Abstract.html.

[2] Jeff Johnson, Matthijs Douze, and Hervé Jégou. “Billion-scale similarity search with GPUs”. In: *IEEE Transactions on Big Data* 7.3 (2019), pp. 535–547. DOI: 10.1109/TBDATA.2019.2921572.

[3] Ziming Mao, Kiran Srinivasan, and Anurag Khandelwal. “TRINITY: A Fast Compressed Multi-attribute Data Store”. In: *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys ’24)*. ACM, 2024, pp. 405–420. DOI: 10.1145/3627703.3650072.

[4] NVIDIA Corporation. *CUDA C++ Programming Guide*. Release 13.1. 2026. URL: https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf.

[5] NVIDIA Corporation. *CUDA C++ Best Practices Guide*. Release 13.1. 2026. URL: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/.

[6] Yu A. Malkov and D. A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), pp. 824–836. DOI: 10.1109/TPAMI.2018.2889473.

[7] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. “Product Quantization for Nearest Neighbor Search”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128. DOI: 10.1109/TPAMI.2010.57.

[8] Ruiqi Guo et al. “Accelerating Large-Scale Inference with Anisotropic Vector Quantization”. In: *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 3887–3896.

[9] Muhammad A. Awad et al. “Engineering a High-Performance GPU B-Tree”. In: *Proceedings of the 24th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP 2019)*. ACM, 2019, pp. 145–157. DOI: 10.1145/3293883.3295706.

[10] Martin Koppehel et al. “CuART — A CUDA-Based, Scalable Radix-Tree Lookup and Update Engine”. In: *Proceedings of the 50th International Conference on Parallel Processing (ICPP 2021)*. ACM, 2021, 12:1–12:10. DOI: 10.1145/3472456.3472511.

[11] Xun Zhong et al. “Learned Index on GPU”. In: *Proceedings of the 38th IEEE International Conference on Data Engineering Workshops (ICDEW 2022)*. IEEE, 2022, pp. 117–122. DOI: 10.1109/ICDEW55742.2022.00026.

[12] Kun Zhou et al. “Real-Time KD-Tree Construction on Graphics Hardware”. In: *ACM Transactions on Graphics (SIGGRAPH Asia 2008)*. Vol. 27. 5. ACM, 2008, p. 126. DOI: 10.1145/1409060.1409079.

[13] Julian Labeit, Julian Shun, and Guy E. Blelloch. “Parallel Lightweight Wavelet Tree, Suffix Array and FM-Index Construction”. In: *2016 Data Compression Conference (DCC 2016)*. IEEE, 2016, pp. 33–42. DOI: 10.1109/DCC.2016.40.

[14] Laura Caspari et al. *Beyond Benchmarks: Evaluating Embedding Model Similarity for Retrieval Augmented Generation Systems*. 2024. arXiv: 2407.08275 [cs.IR]. URL: https://arxiv.org/abs/2407.08275.

A Appendix

As described in Chapter 3, the single-threaded GPU algorithm and the CPU baseline algorithm execute identical logic over identical data. Nevertheless, in our study, the single-threaded GPU version consistently performed at least $60\times$ slower than its CPU counterpart. This appendix documents the investigation we conducted to explain that gap.

The first suspect was clock speed. GPU clocks are generally slower than CPU clocks, so for single-threaded workloads a GPU retires fewer instructions per second. As detailed in Section 3.2, however, the CPU used for benchmarking ran at only $1.3\times$ the GPU's SM clock, which cannot account for a $60\times$ slowdown. We therefore wrote detailed benchmarks of both implementations and their constituent parts.

We first checked whether the slowdown was caused by host-side overhead associated with launching the GPU kernel. As shown in Figure A.1, the kernel itself dominated end-to-end time at every scale, although device-to-host copyback became non-trivial at larger N. Kernel launch and allocation were negligible.

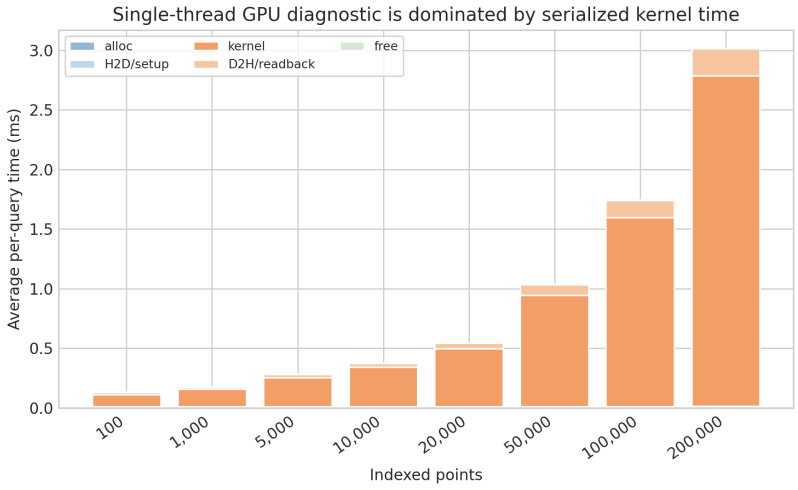


Figure A.1: End-to-end breakdown of single-thread GPU time per query versus indexed points, split into allocation, H2D/setup, kernel execution, D2H readback, and free. The kernel itself is the dominant cost at every N; launch and allocation overheads are effectively constant and do not explain the observed scaling.

Next, we instrumented the kernel itself to see where time was spent inside a single-query traversal. As shown in Figure A.2, most of the kernel time was spent in node processing, and within that, the collapsed-node path dominated—consistent with the fact that collapsed nodes substantially outnumber uncollapsed ones in our tries.

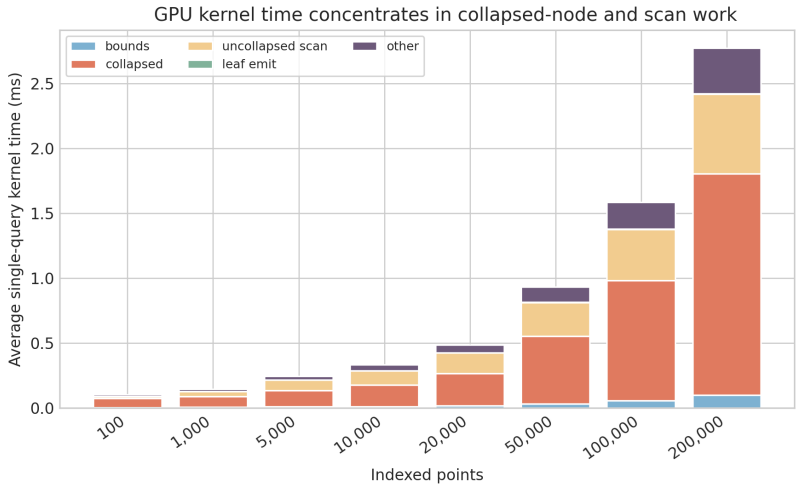


Figure A.2: Per-query kernel time broken down by phase: bounds arithmetic, collapsed-node handling, uncollapsed bitmap scans, leaf emission, and residual work. Collapsed-node handling and uncollapsed scan dominate at large N.

We noticed that the uncollapsed scan relied on the `__ffsll` (find-first-set on a 64-bit word) instruction. To test whether this specific instruction was unusually expensive on our GPU, we built a microbenchmark that loaded data into memory and then repeatedly applied `__ffsll` to it. For each substrate (CPU and GPU), we also authored a baseline kernel in which every step was identical except that `__ffsll` was replaced by an integer addition. We used compiler directives to prevent optimization from collapsing the loops. We swept input sizes and averaged over five trials, then compared the GPU slowdown (ratio of `__ffsll` time to `add` time) to the equivalent CPU ratio. Figure A.3 shows the result: the isolated `__ffsll` instruction is roughly $10\times$ more expensive on the GPU relative to its local add baseline than on the CPU. This confirms that, beyond clock speed, certain complex bit-manipulation instructions carry a disproportionate cost on the GPU.

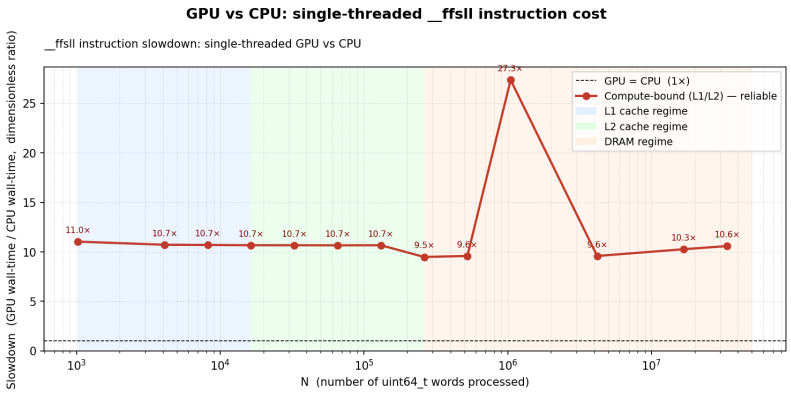


Figure A.3: GPU vs. CPU cost of the `__ffsll` instruction, normalized against an `add`-only baseline on each substrate, as a function of the number of 64-bit words processed. The GPU pays roughly $10\times$ the relative penalty for `__ffsll` that the CPU does in the compute-bound regime.

While the `__ffsll` gap partially accounts for the observed discrepancy, it does not explain the full $60\times$ slowdown. Finally, we investigated the cost of memory accesses themselves. We built a benchmark that issued many sequential 64-byte loads, analogous to the access pattern a traversal step performs when fetching the next node's bitmap.

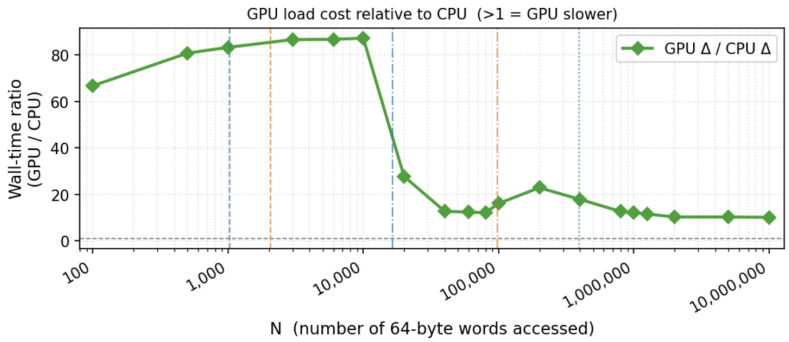


Figure A.4: Ratio of GPU to CPU wall-time for a sequential-load microbenchmark, as a function of the number of 64-byte words accessed. Values above one indicate the GPU is slower. The GPU is dramatically slower for working sets that fit in cache, and the ratio narrows only once the access stream is large enough to saturate DRAM bandwidth on both substrates.

As shown in Figure A.4, memory-access time is considerably slower on the GPU for the small-to-medium working sets that characterize a single-threaded trie traversal. This is consistent with the architectural baseline: an L1 hit costs 1–3 cycles on a modern CPU but up to $\sim$30 cycles on a GPU.

Taken together, these three experiments support the claim made in the main text: a single GPU thread is not slower than a CPU thread because of arithmetic cost, but because its memory hierarchy is designed to be hidden behind massive parallelism rather than minimized in absolute terms. For a GPU implementation of MDTRIE to outperform the CPU baseline, it must compensate for these per-access costs by exposing enough parallelism to keep the memory subsystem saturated—which is precisely what the optimized kernel in Section 3 is designed to do.